Using neural network for credit card fraud detection

Cite as: AIP Conference Proceedings **2159**, 030013 (2019); https://doi.org/10.1063/1.5127478
Published Online: 02 October 2019

Sevdalina Georgieva, Maya Markova, and Velizar Pavlov

Using Neural Network for Credit Card Fraud Detection

Sevdalina Georgieva[1, a)], Maya Markova[1, b)] and Velizar Pavlov[1, c)]

[1] *University of Ruse, 8 Studentska Street, 7017 Ruse, Bulgaria*

[a)] sevdalina095@abv.bg
[b)] maya.markova@gmail.com
[c)] vpavlov@uni-ruse.bg

Abstract. In the present time, the internet has become more popular in almost every domain of life. Online shopping, e-commerce and transactions are increasing every day. Fraudulent activities have also increased in the online payment system. Payment card fraud has become a serious problem in the world. Companies and institutions lose huge amounts annually due to fraud and the fraudsters continuously seek new ways to commit illegal actions. The good news is that fraud tends to be perpetrated to certain patterns and that it is possible to detect such patterns, and hence fraud. In this paper, we present a way to detect fraudulent transactions by a neural network model. Artificial neural network (ANN), when trained properly can work like a human brain. They learn by example, like people and are known as a very good classifier. Among the main characteristics of credit card traffic are the great imbalance between proper and fraudulent operations. At the same time, public data are hardly available for confidentiality issues. To deal with the imbalanced dataset we use resampling techniques. To ensure proper model construction we made a pattern recognition network trained with a scaled conjugate gradient backpropagation algorithm using the *Neural network toolbox* in Matlab.

INTRODUCTION

Nowadays more and more people prefer to use credit cards when purchasing goods and services [1]. Credit cards are very useful when shopping online as well as when paying bills and taxes online. Not only that they are convenient, but they also save time. Many people find credit card payment in stores much more practical than cache payments. However, with the increase in the number of credit card users, the number of credit card frauds also increases. Neural network applications have great potential to detect and prevent fraud.

The goal of credit card fraud detection is to decide whether a transaction is fraudulent based on historical data. It is not easy to decide because of changes in client spending practices, especially during the holiday season. Fraudsters use different techniques to overcome fraud protection. It is currently realized that artificial neural networks offer a successful way to deal with problems like these [2].

According to Pozzolo et al. [3], one of the most explored fields of fraud detection [4, 5, 6] is credit card fraud detection. It detects fraudulent behavior by automated analysis of previous transaction data. Many attributes (e.g. recipient, quantity of the transaction, date of transaction, credit card identifier) are stored in the databases for every transaction. But fraud occurrence could be detected only of the credit card level because one transaction is not sufficient [4]. All transactions for one credit card are grouped for learning behavioral models. The data for each card is analyzed to recognize fake transactions [7].

Because there is a large volume of customer data and transactional data, neural networks can be effectively used for the detection of credit card behavior and usage patterns. Neural network applications can detect and prevent fraud by analyzing the irregularities in the pattern changes.

Sixth International Conference on New Trends in the Applications of Differential Equations in Sciences (NTADES'19)
AIP Conf. Proc. 2159, 030013-1–030013-10; https://doi.org/10.1063/1.5127478
Published by AIP Publishing. 978-0-7354-1904-9/$30.00

CREDIT CARD FRAUD

Credit card fraud can be defined as the usage of a credit card by someone who is not the owner of the card and is not authorized to use it. Specifically, such fraud is carried through when an individual uses someone's credit card for personal purposes without the knowledge and permission of the owner.

The easiest way to commit physical fraud without using technology is by theft. Another way is by using fake and counterfeit credit cards.

Some frauds are done and operate from a web site that offers products and goods at heavily discounted rates. When the clients place an order they need to give personal credit card details. Then the fraudsters use this information to buy the ordered products on regular prices and other goods that then they sell on their site. The clients receive the placed order and might not even realize that their card is being used without their knowledge. This type of fraud is called triangulation.

Phishing is a well-known type of fraud. This method is designed to obtain sensitive information such as user's name, password and credit card details. Usually, this happens via spam e-mails that look like they come from a website or company the user trusts.

CREDIT CARD FRAUD AND NEURAL NETWORKS

Credit card fraud detection using neural networks relies on the human brain operating principal. Neural network technology can make a computer capable of thinking. Credit card fraud detection with a neural network is possible due to the ability of the neural network to solve classification problems. The human brain learns through experience. This technology works similarly. When customers use a credit card, they create a fixed pattern. This pattern is later used in the classification.

The neural network is trained to detect credit card fraud by using historically collected data. This data contains information such as the card holder’s profession, income, credit card number, transactions of larger amount of money, frequency of transactions, location of where purchases take place, information about the kind of purchases made before with the credit card (e.g. grocery store, gas stations, hotels, specific restaurants, etc.), issue date of the card. Using this data the neural network decides whether a particular transaction is being made by the cardholder or not.

When a credit card is being used, the system compares the information about the transaction with the information saved during previous uses. If the data matches the pattern, then it is almost certain that the card is used by its owner. If there isn't any match, that doesn't mean certain fraud, but the possibilities for fraud increase significantly.

Of course, there are cases in which a transaction made by the cardholder is sort of completely different than previous transactions and cases in which the info matches the pattern however the transaction isn't made by the cardholder. However, the first is not frequent and in the worst case, the only consequence would be that the transaction won’t be completed.

Fraud detection in credit card operation is a pattern recognition problem. However, it has certain characteristics of its own that make it rather difficult. The most important is the great imbalance between good operations and fraudulent ones.

HANDLING IMBALANCED DATASETS

An imbalanced dataset is one where the number of observations belonging to one group or class is significantly higher than those belonging to the other classes.

Machine Learning algorithms [8] are very likely to produce faulty classifiers when they are trained with imbalanced datasets. These algorithms tend to show a bias for the majority class, treating the minority class as a noise in the dataset. With many standard classifier algorithms, there is a likelihood of the wrong classification of the minority class.

There is also a problem of vanity metrics in measuring the performance of algorithms on imbalanced datasets. If we have an imbalanced dataset containing 1% of a minority class and 99% of the majority class, an algorithm can predict all cases as belonging to the majority class. The accuracy score of this algorithm will yield an accuracy of 99% which seems impressive, but is it really? The minority class is totally ignored in this case and this can prove expensive in some classification problems, such as the case of a credit card fraud, which can cost individuals and businesses lots of money.

Since accuracy is a poor measure for performance on unbalanced datasets, other indicators are used. To establish this, let us first define some terms:

False Positive (FP): This is used to describe positive predictions that are actually negative in real life. An example will be predicting a credit card transaction is fraudulent when in truth it is not.

True Positive (TP): This is used to describe positive predictions that are actually positive in real life.

False Negative (FN): This is used to describe negative predictions that are actually positive in real life. An example will be predicting a credit card transaction is not fraudulent when in truth it is.

True Negative (TN): This is used to describe negative predictions that are actually negative in real life.

Some Measures and indicators of model performance include:

- **Precision:** This is an indicator of the number of items correctly identified as positive out of total items identified as positive. The formula is given as:

$$\textbf{Precision} = \mathrm{TP}/(\mathrm{TP} + \mathrm{FP})$$

- **Recall / Sensitivity / True Positive Rate (TPR):** This is an indicator of the number of items correctly identified as positive out of total actual positives. The formula is given as:

$$\textbf{Recall} = \mathrm{TP}/(\mathrm{TP} + \mathrm{FN})$$

Precision can be seen as "how useful the results are", and recall is "how complete the results are"

- **Specificity / True Negative Rate (TNR):** This is an indicator of the number of items correctly identified as negative out of total actual negatives. The formula is given as:

$$\textbf{Specificity} = \mathrm{TN}/(\mathrm{TN} + \mathrm{FP})$$

- **F1 Score:** This is a performance score that combines both precision and recall. It is a harmonic mean of these two variables. The formula is given as:

$$\textbf{F1} = 2 * \mathrm{Precision} * \mathrm{Recall}/(\mathrm{Precision} + \mathrm{Recall})$$

- **Matthew Correlation Coefficient (MCC) Score:** This is a performance score that takes into account true and false positives, as well as true and false negatives. This score gives a good evaluation for imbalanced dataset. The formula is given as:

$$\mathrm{MCC} = \frac{\mathrm{TP} * \mathrm{TN} - \mathrm{FP} * \mathrm{FN}}{\sqrt{(\mathrm{TP} + \mathrm{FP}) * (\mathrm{FN} + \mathrm{TN}) * (\mathrm{TP} + \mathrm{FN}) * (\mathrm{FP} + \mathrm{TN})}}$$

- **The area under the ROC Curve:** An ROC curve (receiver operating characteristic curve) is a graph that shows the performance of a classification algorithm at all classification thresholds. This curve plots TPR vs. FPR.
- **The area under the Precision-Recall Curve:** A Precision-Recall curve is a graph that shows performance by plotting precision against the recall.
- **Confusion Matrix:** This is a graphical representation of the TP, FP, FN and TN. A generalized confusion matrix is given below in Fig.1.

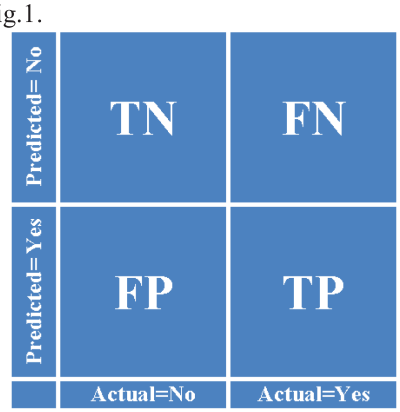


Figure 1 Generalized confusion matrix

Generally, the best measures for imbalanced datasets are Matthew Coefficient Correlation Score, F1 Score and The Area under the Precision-Recall Curve.

Still, it is important to understand the business implication of the model when looking to choose a performance evaluation metric. Ideally, there would be trade-offs between false positives and false negatives in real life. When we make classifying against credit card fraud, we prefer to have a false positive than have a false negative. That is, we would rather predict that a transaction is fraudulent when it actually is not than predict that a transaction is not fraudulent when it is.

PREPARING THE DATASET

The dataset is obtained from web site kaggle.com [9]. It contains transactions made by credit cards in September 2013 by European cardholders. This dataset presents transactions that occurred in two days, where we have 492 frauds out of 284 807 transactions. The dataset is highly unbalanced, the positive class (frauds) account for 0.172% of all transactions.

It contains only numerical input variables which are the result of a Principal component analysis[1] (PCA) transformation. Unfortunately, due to confidentiality issues, we cannot receive the original features and more background information about the data. Features V1, V2, ... V28 are the principal components obtained with PCA, the only features which have not been transformed with PCA are 'Time' and 'Amount'. Feature 'Time' contains the seconds elapsed between each transaction and the first transaction in the dataset. The feature 'Amount' is the transaction amount. Feature 'Class' is the response variable and it takes value 1 in case of fraud and 0 otherwise.

How different is the amount of money used in different transaction classes? We calculate descriptive statistics for the amount using the IBM SPSS Statistics software [10]. The mean value of all transactions is $88.35 while the largest transaction recorded in this data set amounts to $25 691.16. The mean and standard deviation for normal and fraud data are shown in Fig. 2. As could be seen from percentiles 50% of fraud transactions are below 10$, but there is 25% of transactions above 100$. Maximum of fraud transaction is 2 125.87$.

Class = Normal

Statistics[a]

Amount

N	Valid	284315
	Missing	0
Mean		88,2910
Std. Deviation		250,10509
Minimum		,00
Maximum		25691,16
Percentiles	25	5,6500
	50	22,0000
	75	77,0500

a. Class = Normal

Class = Fraud

Statistics[a]

Amount

N	Valid	492
	Missing	0
Mean		122,2113
Std. Deviation		256,68329
Minimum		,00
Maximum		2125,87
Percentiles	25	1,0000
	50	9,2500
	75	105,8900

a. Class = Fraud

Figure 2 Descriptive statistics for the amount of normal and fraud transactions

[1] Principal component analysis (PCA) is a statistical procedure that uses an orthogonal transformation to convert a set of observations of possibly correlated variables (entities each of which takes on various numerical values) into a set of values of linearly uncorrelated variables called principal components.

Let's have a more graphical representation of the amount per transactions in fig. 3. From graphics for fraud transaction, could be seen that there are pics for the amount of 1$ and 100$. This is because fraudsters usually make $1 transaction first to check the card and then withdraw $100 or more.

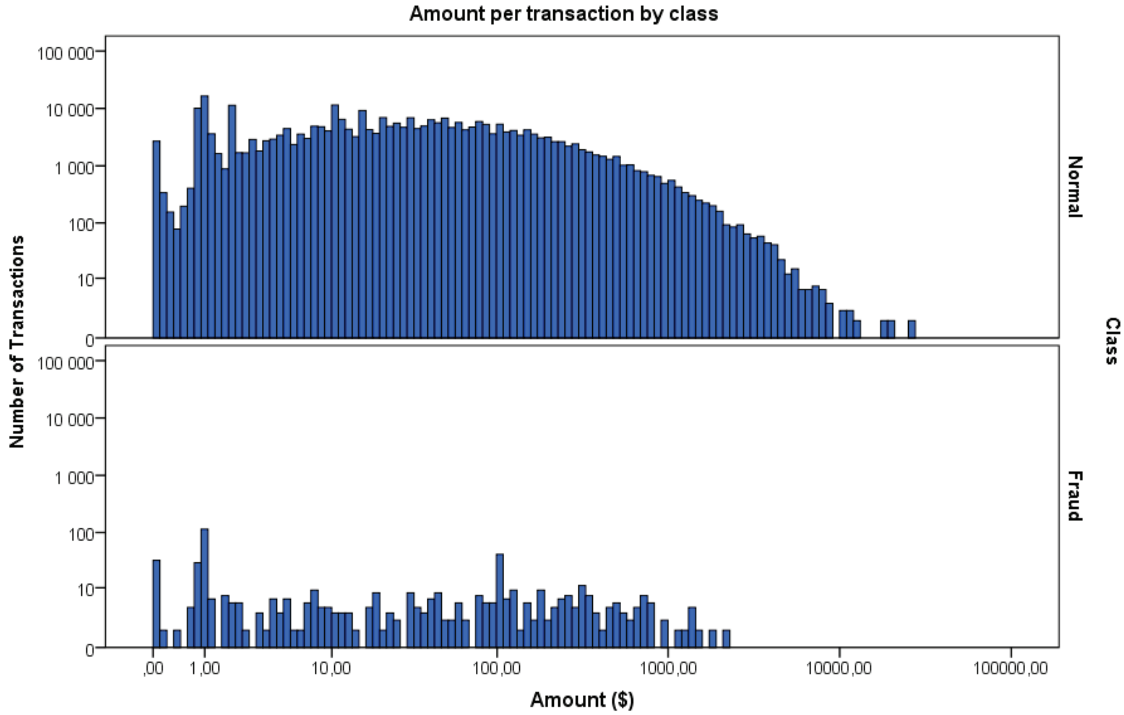


Figure 3 Amount per transaction by class

The number of fraud transactions in real life is much less than non-fraud cases, which is reflected exactly in this dataset. In fact, 99.83% of the transactions in this data set were not fraudulent while only 0.17% were fraudulent. The following visualization (fig.4) underlines this significant contrast.

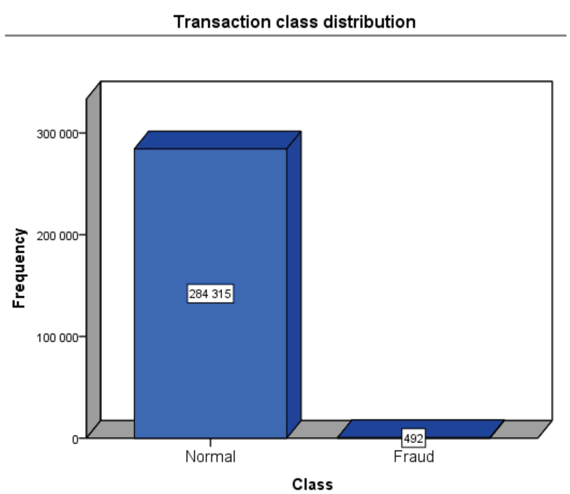


Figure 4 Transaction class distribution

Figure 5 Neural network with the unbalanced dataset results

We made a pattern recognition neural network and we trained the network with the unbalanced dataset. Figure 5 shows the confusion matrices for training, testing, and validation, and the three kinds of data combined. The lower right blue squares illustrate the overall accuracies. For all confusion matrix accuracies is 99.9%. The precision score is 87.5%, the recall score is 78.3%, the F1 score is 82.68% and Matthew Coefficient score is 82.72%. Notice that the accuracy is very high, despite the presence of false positives and false negatives. However, the F1 score and MCC score tell a better story of the actual model performance. That is why we proceeded to balance the dataset.

There are various techniques implemented for dealing with the imbalanced dataset. Some popular strategies include improving the classification algorithm to fit better with the imbalanced dataset or balancing the classes of training data before providing data as input to the algorithm (data resampling techniques). In this paper, we use the second technique.

Some popular resampling techniques including:

- Random undersampling
- Random oversampling
- SMOTE (Synthetic Minority Over-sampling Technique)

We use under-sampling techniques (fig.6).

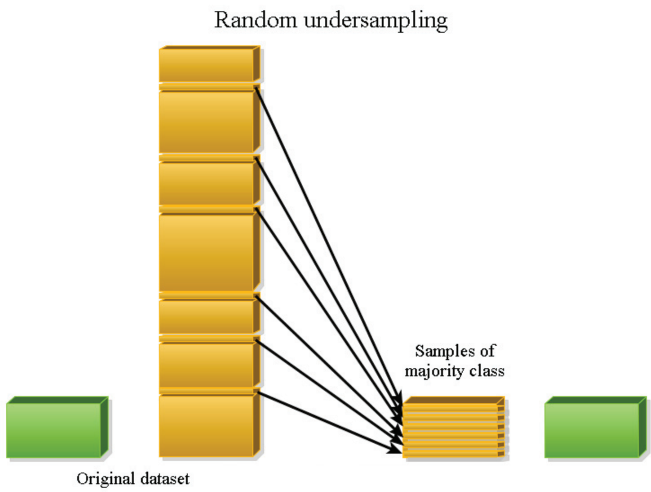


Figure 6 Random undersampling

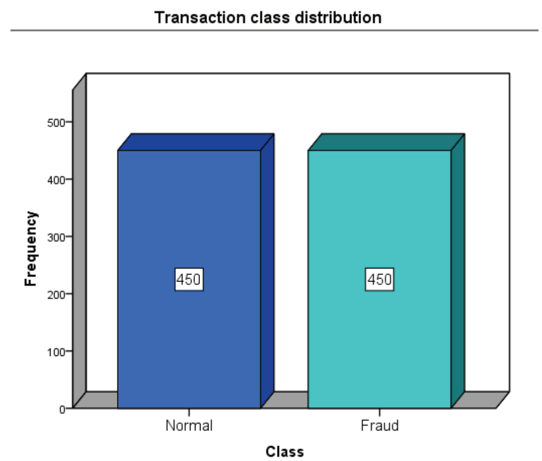


Figure 7 Transaction class distribution of the balanced dataset

We take a subset of 900 transactions from the dataset. By removing random records from the majority class we receive subset which contains 450 normal and 450 fraud transactions (fig. 7).

We use supervised learning for training the neural network. Supervised techniques [1] depend on the set of past transactions for which is known as a label (also called class) of the transaction. In credit card fraud detection tasks, there are two types of labels genuine (the transaction was made by the cardholder) or fraudulent (the transaction was made by a fraudster). The data for defining the label is from a customer complained or as a result of an investigation by the credit card company. Supervised techniques use labeled past transactions to create a fraud prediction model that returns, for any new transaction, the likelihood of it being a fraud.

BUILDING THE MODEL

There are infinite numbers of ways to construct a neural network. The number of input neurons is one of the easiest parameters to select once the independent variables are selected because each independent variable is represented by its input neuron.

We use pattern recognition networks with 29 input neurons - one neuron for every feature from ’V1’ to ’V28’ and one for ‘Amount’ feature. We removed ’Time’ from the dataset because we did not use it for any of our analyses. Feature 'Class' was used as target data. Pattern recognition networks are feedforward networks that can be trained to classify inputs according to target classes.

Hidden layers provide the network with its ability to generalize [11]. In practice, neural networks with one and occasionally two hidden layers are widely used and have performed very well. Increasing the number of hidden layers also increases the computation time and the danger of overfitting, which leads to poor out-of-sample forecasting performance. Here we analyzed neural network structure with one hidden layer.

Determining the number of neurons in the hidden layer is more difficult. Despite its importance, there is no ”magic" formula for selecting the optimum number and therefore researchers fall back on experimentation. We use 10 to 15 neurons in the hidden layer and the network with 13 neurons gives the best results (fig. 8).

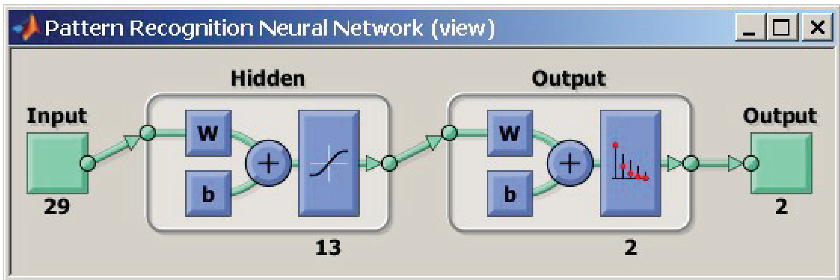


Figure 8 Architecture of trained pattern recognition neural network

Deciding on the number of output neurons is somewhat more straightforward since there are compelling reasons to always use one or two output neurons. We use two output neurons.

When training multilayer networks, the general practice is to first divide the data into subsets. The input vectors and target vectors are randomly divided into three sets.

The first subset 70% of data is the training set, which is used for computing the gradient and updating the network weights and biases.

The second subset 15% of data is the validation set, used to validate that the network is generalizing and to stop training before overfitting

The last test subset 15% of data is used as a completely independent test of network generalization.

Once the network weights and biases are initialized, the network is ready for training. The training process requires a set of examples of proper network behavior—network inputs *p* and target outputs *t*.

The network is trained with Scaled Conjugate Gradient (SCG) backpropagation algorithm. SCG [12] is a supervised learning algorithm used for feedforward neural networks and the member of the class of conjugate gradient methods. SCG uses similar concepts of the general optimization strategy, but chooses the search direction and step size more effectively by using information from the second-order approximation is represented by this equation

$$E(w+y) \approx E(w) + E'(w)^T + \frac{1}{2} y^T E''(w) y \,.$$

In SCG, each iteration computes optimal distance. The line search is then performed to determine the optimal distance to move along the current search direction as equation

$$w_{k+1} = w_k + a_k * p_k \,.$$

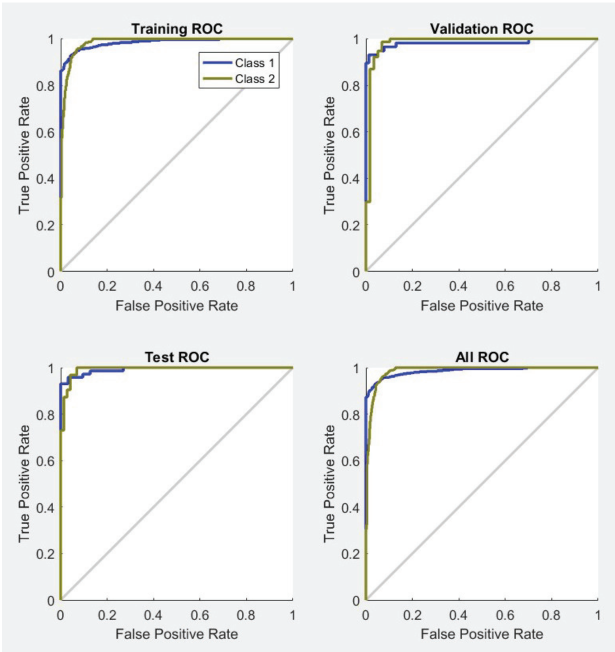
Then the next search direction is performed so that, the conjugated to previous search instructions. Actually, p_k is a function of a_k, the Hessian matrix of the error function and also the matrix of the second derivatives. SCG uses a scalar a_k that is supposed to regulate the indefiniteness of the Hessian matrix.

The next figure (Fig.9) shows the confusion matrices for training, testing, and validation, and the three kinds of data combined. The network outputs are very accurate, as you can see by the high numbers of correct responses in the green squares and the low numbers of incorrect responses in the red squares. The lower right blue squares illustrate the overall accuracies. For all confusion matrix accuracies is 94.2%. The precision score is 91.6%, the recall score is 97.3%, the F1 score is 94.40% and Matthew Coefficient score is 88.62%. With balanced data, accuracy is almost equal to the F1 score. The F1 score and MCC score show that this model is much better than the former one.

In our model false negative (predict that a transaction is not fraudulent when it is) results are only 1.3%. False positive (predict that a transaction is fraudulent when it actually is not) results are 4.4% but this is a minor problem because it will only prevent the cardholder from making the payment at this moment.

Figure 9 Confusion matrices for trained network

Receiver Operating Characteristic (ROC) curve (fig. 10) is a plot of the true positive rate (sensitivity) versus the false positive rate (1 - specificity) as the threshold is varied. A perfect test would show points in the upper-left corner, with 100% sensitivity and 100% specificity. For this problem, the network performs very well.



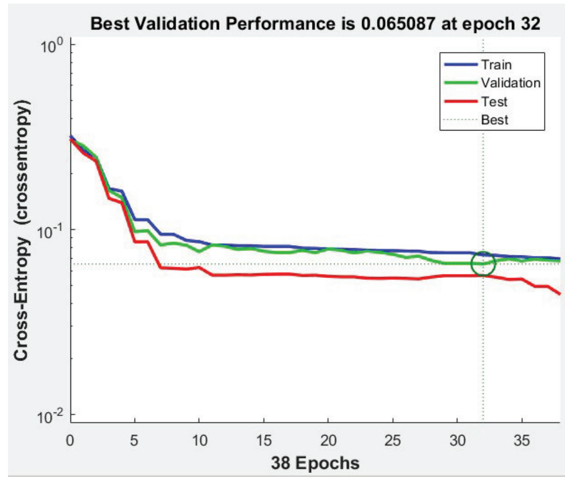


Figure 10 Receiver Operating Characteristic plot

Figure 11 Performance plot

The performance function for the pattern recognition network is Cross-entropy. This performance function is recommended for classification and pattern recognition problems. It calculates a network performance with a measure of the crossentropy of estimated and actual class memberships. The cross-entropy for each pair of output/target elements is calculated as:

$$ce = -t.* \log(y)$$

The aggregate cross-entropy performance is the mean of the individual values:

$$perf = sum(ce(:))/numel(ce)$$

In fig. 11 is shown the performance plot of three data subsets. If the performance on the training set is good, but the test set performance is significantly worse, which could indicate overfitting, then reducing the number of neurons can improve your results. In our case test set performance is better than train and validation sets which show that the network is well trained.

CONCLUSIONS

The advantage of detecting credit card fraud using a neural network is that they could detect credit card behavior and usage patterns in a large volume of customer and transactional data. Just like the human brain recognizes a person by their actions and gestures the same way the neural network recognizes whether or not a credit card is being used by its owner by the card's history. The biggest disadvantage when applying it is the problem with an imbalanced dataset because the number of fraud transactions in real life is much less than non-fraud cases. This problem is partially solved by the data resampling technique. For future works could be used improving classification algorithm to fit better with imbalanced dataset.

In this paper, we make a neural network model that makes the classification of credit card transactions based on real historical data. Pattern recognition networks with 29 input neurons, one hidden layer with 13 neurons and one output layer with 2 neurons are made with Neural network toolbox in Matlab. The network accuracy is 94.2%. The precision score is 91.6%, the recall score is 97.3%, the F1 score is 94.40% and Matthew Coefficient score is 88.62%. With balanced data, accuracy is almost equal to the F1 score. The F1 score and MCC score show that the model is working very good.

REFERENCES

1. F. Carcillo, Y.- A. Le Borgne and O. Caelen et al., Combining unsupervised and supervised learning in credit card fraud detection, Information Sciences, https://doi.org/10.1016/j.ins.2019.05.042
2. A. Dal Pozzolo, O. Caelen, G. Bontempi, When is undersampling effective in unbalanced classification tasks? in: Joint European Conference on Machine Learning and Knowledge Discovery in Databases, Springer, 2015, pp. 200–215.
3. Andrea Dal Pozzolo, Olivier Caelen, Yann-Ael Le Borgne, Serge Waterschoot, Gianluca Bontempi. Learned lessons in credit card fraud detection from a practitioner perspective, Expert systems with applications,41,10,4915-4928,2014, Pergamon
4. R. J. Bolton, D. J. H and, et al., Unsupervised profiling methods for fraud detection, Credit Scoring and Credit Control VI I (2001) 235–255.
5. R. Brause, T. Langsdorf, M. Hepp, Neural data mining for credit card fraud detection, in Tools with Artificial Intelligence, 1999. Proceedings., IEEE, 1999, pp. 103–106.
6. P. Chan, W. Fan, A. Pro dromidis, S. Stolfo, Distributed data mining in credit card fraud detection, Intelligent Systems and their Applications 14 (1999) 67–74.
7. C. Whitrow, D. J. Hand, P. Juszczak, D. Weston, N. M. Adams, Transaction aggregation as a strategy for credit card fraud detection, Data Mining and Knowledge Discovery 18 (2009) 30–55.
8. https://medium.com/coinmonks/handling-imbalanced-datasets-predicting-credit-card-fraud-544f5e74e0fd
9. https://www.kaggle.com/mlg-ulb/creditcardfraud/home
10. V. Pavlov and V. Mihova, Applied Statistics with SPSS (Avangard Print, Ruse, 2016). [in Bulgarian]
11. Leandro S. Maciel, Rosangela Ballini, Neural Networks Applied To Stock Market Forecasting: An Empirical Analysis, Learning and Nonlinear Models (L&NLM) –Journal of the Brazilian Neural Network Society, Vol. 8, Iss. 1, pp. 3-22, 2010.
12. Sasmita Nayak, Dr. Neeraj Kumar, Dr. B. B. Choudhury, Scaled Conjugate Gradient Backpropagation Algorithm for Selection of Industrial Robots, International Journal of Computer Application (2250-1797), Volume 7– No.6, November-December 2017, https://dx.doi.org/10.26808/rs.ca.i7v6.12